

Name : Moomna Atiq

Registration no : 23-BCS-065

Course : DevOps

Department : CS

Semester : 7th(2023-2027)

Submitted to : Mr Waqas Saleem

Lab#04

Lab 4: Simple Python Database Connections

Student GitHub Username: Moomnaatiq808

Ubuntu Identity: Moomnaatiq808@ubuntu

Hostname: ubuntu

Databases Fork URL: https://github.com/Moomnaatiq808/Databases

DevOps Lab Directory URL:

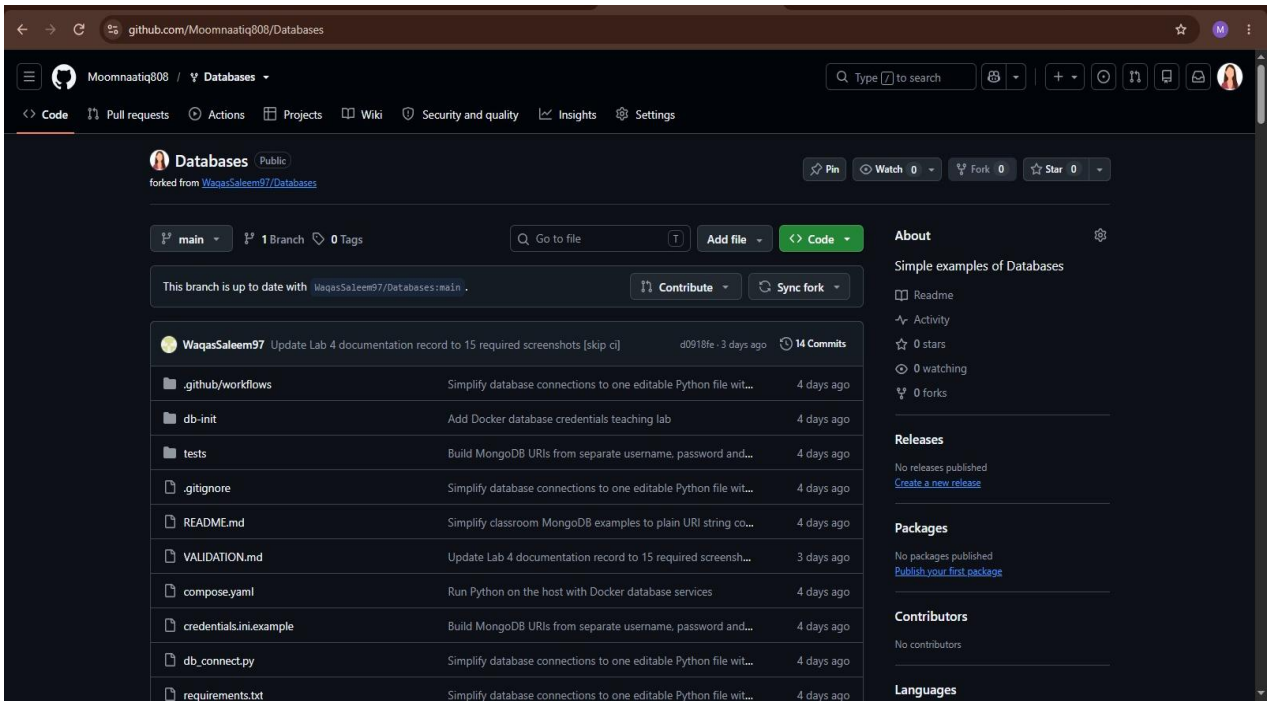
https://github.com/Moomnaatiq808/DevOps/tree/main/Labs/Lab04

Lab 4: Simple Python Database Connections

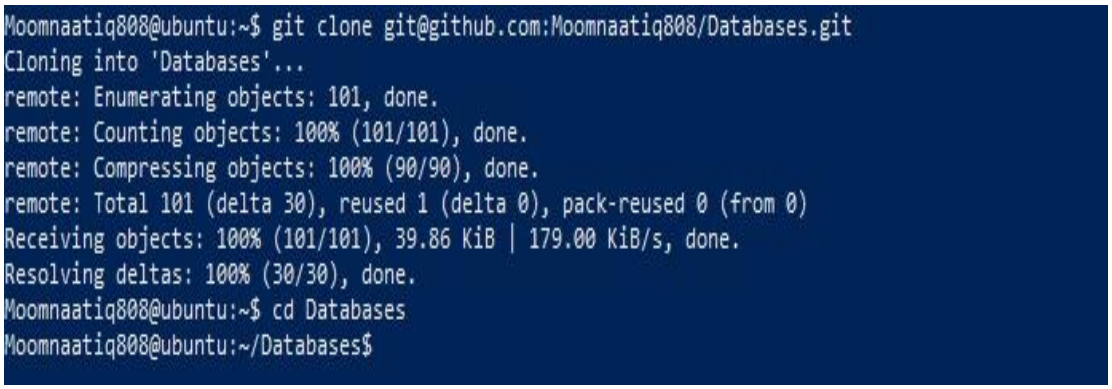
Task 1: Fork and Clone the Databases Repository

- I opened the instructor's Databases repository and forked it under my account, keeping the name Databases.
- The page shows <username>/Databases and the original repository as the source. ***(Figure 1: databases_fork.png)***
- I cloned my own fork to the Ubuntu server using SSH and entered the directory. ***(Figure 2: databases_clone.png)***

databases_fork.png



databases_clone.png



Task 2: Install Docker, Docker Compose, and Python on Ubuntu

- I updated the package index and installed docker.io and curl.
- I downloaded the Docker Compose plug-in into the system CLI plug-ins directory and made it executable.
- I started Docker and added my user to the docker group, then logged out and reconnected for the group change to apply.
- I verified Git, Docker and Docker Compose versions. ***(Figure 3: docker_installation.png)***
- I installed Python 3, created a virtual environment **.venv,** activated it and installed **requirements.txt** .

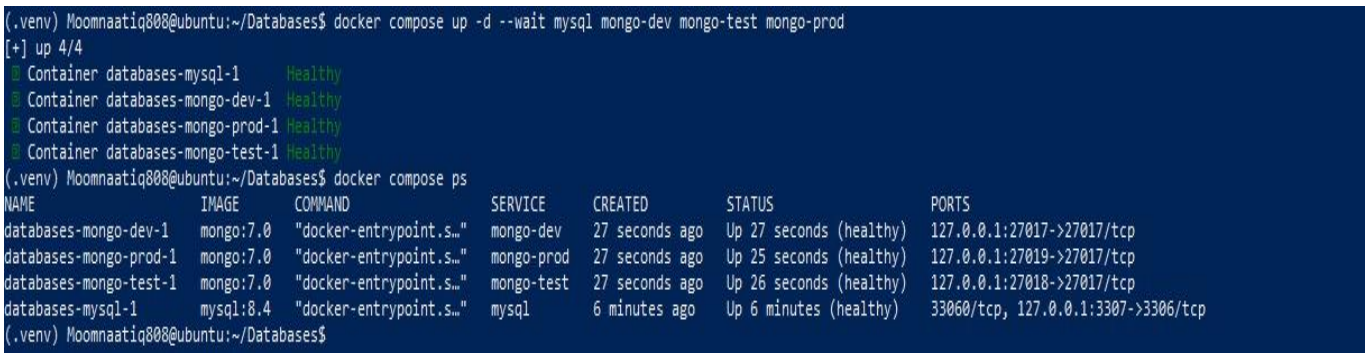
docker_installation.png



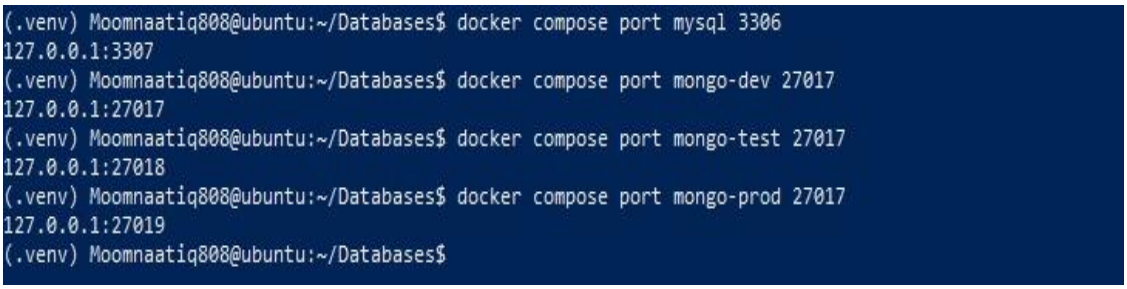
Task 3: Start the Database Services

- I started MySQL, MongoDB dev, MongoDB test and MongoDB prod containers with **Docker Compose.**
- docker compose ps shows all four services as healthy. ***(Figure 4: database_services.png)***
- I checked the host ports: MySQL 3307, mongo-dev 27017, mongo-test 27018, mongo-prod 27019. ***(Figure 5: database_ports.png)***

database_services.png



database_ports.png



Task 4: MySQL with Values in the Code

- **db_connect.py** uses mysql.connector.connect() with host, port, user, password and database written directly in code.
- host and port identify the server, user and password are the credentials, and database selects the database.
- The program ran a query on environment_info to read the environment value.

- **Result:** Connected to MySQL: school_mysql, Server: 127.0.0.1 3307, Environment: dev. ***(Figure 6: mysql_hardcoded_connection.png)***
- The cursor and connection were closed to release resources.

mysql_hardcoded_connection.png

```
(.venv) Moomnaatiq808@ubuntu:~/Databases$ cd ~/Databases
(.venv) Moomnaatiq808@ubuntu:~/Databases$ nano db_connect.py
(.venv) Moomnaatiq808@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) Moomnaatiq808@ubuntu:~/Databases$
```

Task 5: MongoDB with Values in the Code

- I replaced the file with a PyMongo program.
- Variables username, password, endpoint and database_name hold the settings, and the + operator joins them into the URI.
- MongoClient(uri) creates the client and client[database_name] selects the database.
- database.command("ping") sends a real request to confirm connectivity.
- serverSelectionTimeoutMS=5000 limits server selection to 5 seconds.
- **Result:** database school_dev, port 27017, environment dev. ***(Figure 7: mongo_hardcoded_connection.png)***

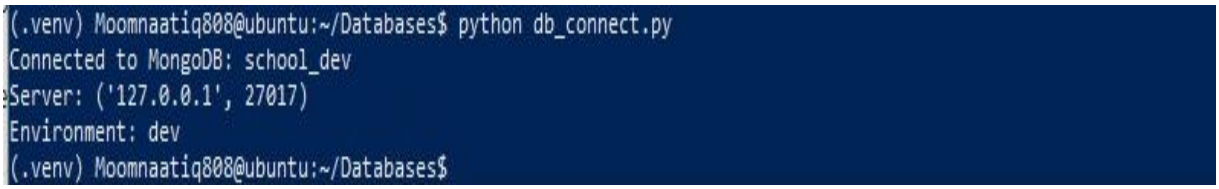
mongo_hardcoded_connection.png

```
(.venv) Moomnaatiq808@ubuntu:~/Databases$ nano db_connect.py
(.venv) Moomnaatiq808@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) Moomnaatiq808@ubuntu:~/Databases$
```

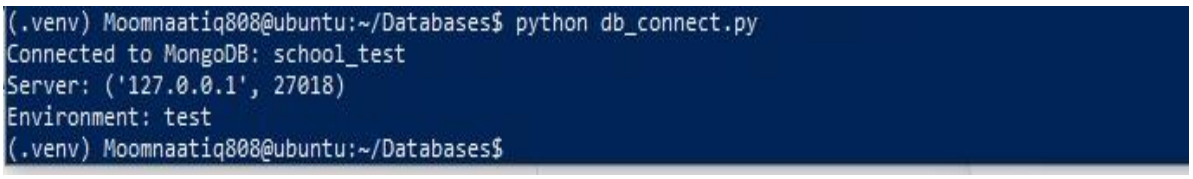
Task 6: MongoDB with os.getenv()

- I changed only the configuration source: the four values are now read with **os.getenv().** The connection block stayed unchanged.
- An if check stops the program if any variable is missing.
- I exported dev values and ran the program: school_dev, 27017, dev. ***(Figure 8: mongo_env_dev.png)***
- I exported test values: school_test, 27018, test. ***(Figure 9: mongo_env_test.png)***
- I exported simulated prod values: school_prod, 27019, prod. ***(Figure 10: mongo_env_prod.png)***
- **clear** was used so that the exports containing credentials were not visible in screenshots.
- **Conclusion:** one unchanged program connects to three databases using process environment values.

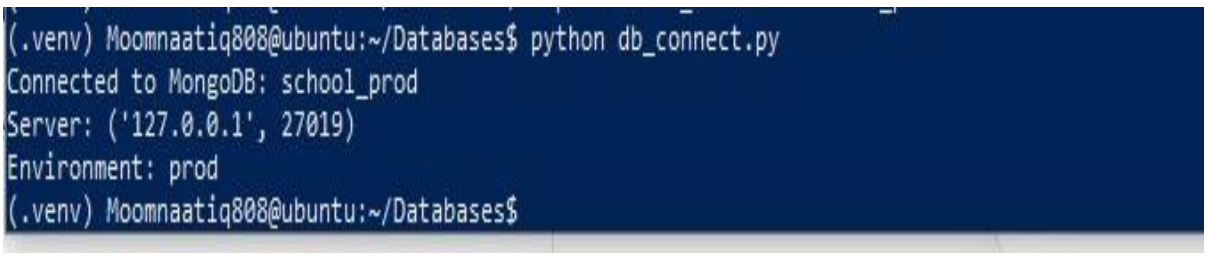
mongo_env_dev.png



mongo_env_test.png



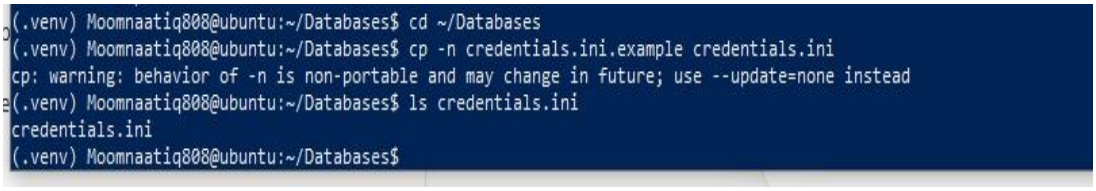
mongo_env_prod.png



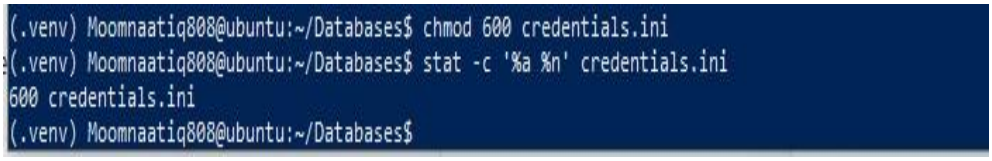
Task 7: MongoDB with a Credentials File

- I copied **credentials.ini**.example to credentials.ini (local file, ignored by Git). ***(Figure 11: credential_files_created.png)***
- I restricted it with chmod 600; stat confirms mode 600. ***(Figure 12: credential_file_permissions.png)***
- The code uses configparser with interpolation=None so values are read literally.
- APP_ENV selects the section (default dev), and the four settings are read from that section.
- **dev result:** school_dev, 27017, dev. ***(Figure 13: mongo_file_dev.png)***
- **test result:** school_test, 27018, test. ***(Figure 14: mongo_file_test.png)***
- **prod result:** school_prod, 27019, prod. ***(Figure 15: mongo_file_prod.png)***
- The INI file stores plaintext, so file permissions and **.gitignore** are important.

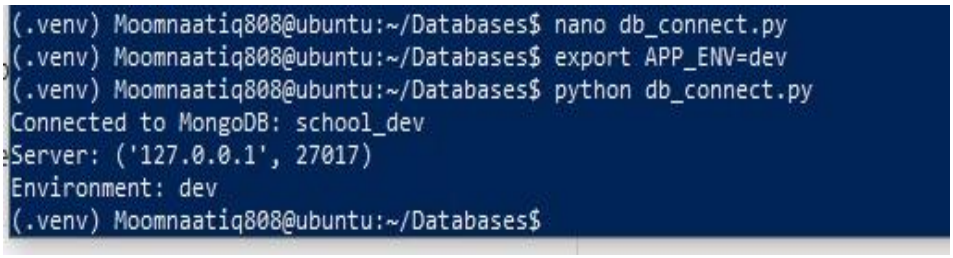
credential_files_created.png



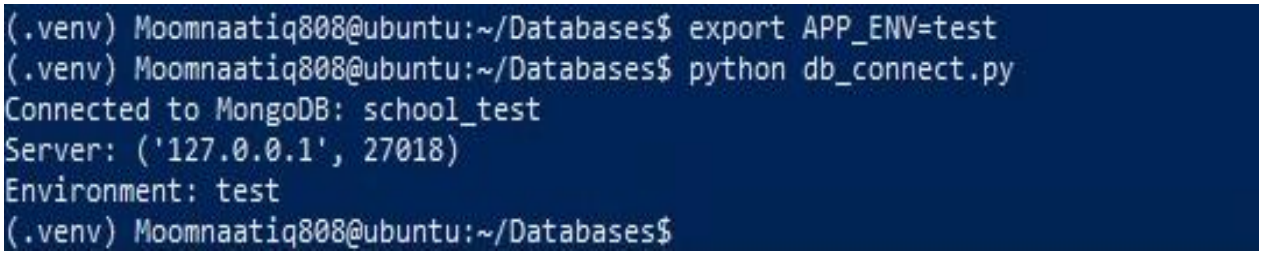
credential_file_permissions.png



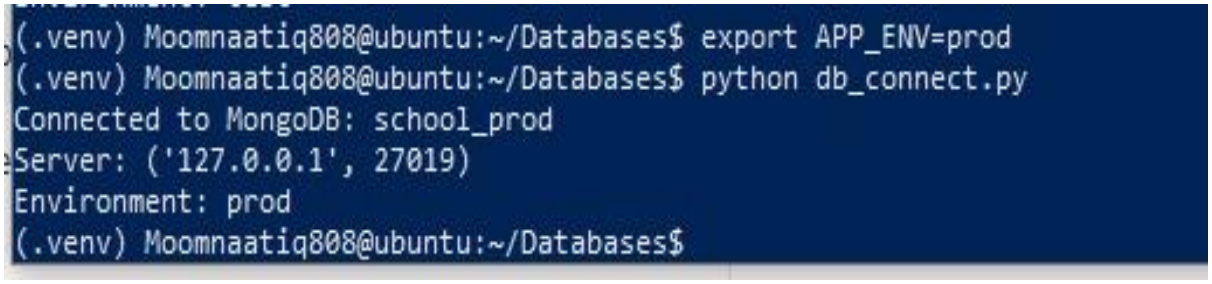
mongo_file_dev.png



mongo_file_test.png



mongo_file_prod.png



Q